

LECTURE 01

Networking Models

From Chaos to a Common Language

Md. Owaliur Rahman

BSc in CSE, BUET

Lecturer, CSE, Presidency University

01

PART ONE

The Case for a Model

Why the real world of communication needs a fixed, agreed-upon way of doing things.

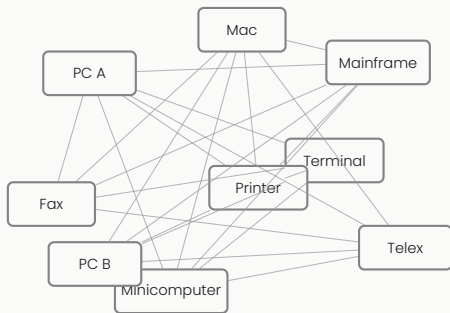
SETTING THE SCENE

Imagine a world without rules

Every device speaks its own dialect. To talk to one other system, you would need to know:

- Its exact cabling and signalling
- Its proprietary data format
- Its own addressing scheme
- Its own error handling rules

Now multiply that by every other device you want to reach.



What we actually need

Instead of every system inventing its own end-to-end solution, we need one thing:

A fixed, shared way of doing things.

Not a single protocol — a model: a common structure everyone agrees to build against, once.

- Break the problem into layers
- Standardise the interfaces
- Design once, reuse everywhere

COMING UP

We will follow a single message from a computer to a server — one layer at a time.

For every layer of the TCP/IP model: the problem it solves, the solution it offers, and what else it quietly does along the way.

02

PART TWO

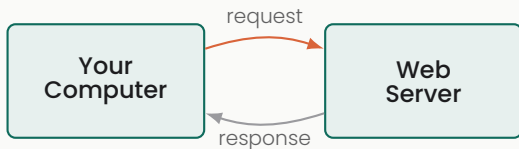
Following a Message

We trace a single request through every layer of the TCP/IP model — down on the way out, up on the way in.

The scenario

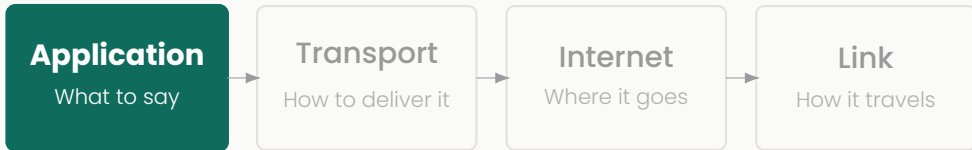
You open a browser and type an address. That one small action has to survive a long trip before a page appears.

To keep it simple, we will follow one request.



A simplified client--server exchange

Where we are: the TCP/IP model



What do you actually want to say?

A person doesn't think in bits. They think:

"Get me this webpage."

"Send this email."

"What's the address for this name?"

But the network below only moves raw bytes. It has no idea what a "webpage" is, or what a request for one should look like.

So the real problem is:

- Every task — browsing, mail, file transfer, name lookup — needs something different
- There is no shared vocabulary between an application and the network
- Software on both ends needs to agree, in advance, on exactly what a request and a response look like

Agree on a protocol per task

Define a precise, shared grammar for each kind of task — an application protocol — that both sides implement identically: exact message structure, exact vocabulary, exact order.

HTTP

Web pages

SMTP

Sending email

FTP

Transferring files

DNS

Looking up names

A simplified HTTP request:

```
GET /index.html HTTP/1.1  
Host: www.example.com  
Accept: text/html
```

Both the browser and the server know exactly how to build this, and exactly how to read it.

What else — and the first package

Beyond just defining requests, this layer also:

- Defines the *data formats* a message may carry — HTML, JSON, images
- Handles session-level details — cookies, login tokens
- Chooses what kind of delivery it needs from the layer below

Whatever the protocol produces is simply handed down as:



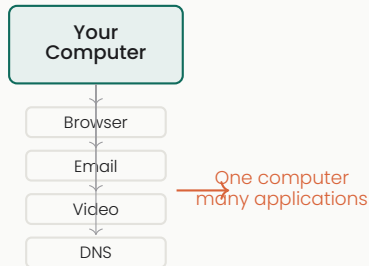
passed down to the Transport layer

No header yet — the Application layer hands over plain data. Encapsulation proper begins next, at the Transport layer.

Who is this data actually for?

The Application layer has produced a message. But a computer can run many applications at the same time:

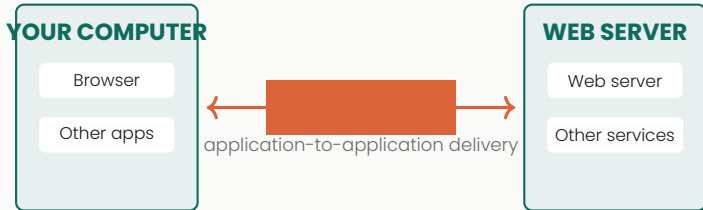
- Browser
- Email client
- Video call
- DNS client



The network can get data to the right computer, but that is not enough.

Transport answers:
Which application should receive this data?

From computer-to-computer to application-to-application



The Transport layer provides communication between applications, not merely between computers.

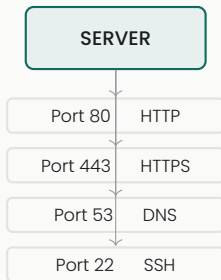
Identify the destination application

An IP address identifies a computer.
But the computer may be running
hundreds of applications
simultaneously.

**IP address = which
computer**

**Port number = which
application**

The Transport layer uses port numbers
to distinguish different application
endpoints



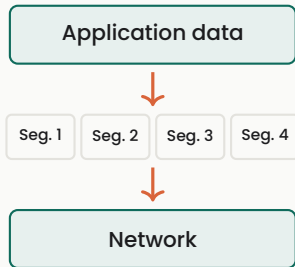
Same computer. Different applications.
Different ports.

Break a large message into manageable pieces

Applications can produce data much larger than a single network packet can conveniently carry.

Transport breaks the application data into smaller pieces.

At the receiving side, those pieces can be put back together before the data reaches the application.

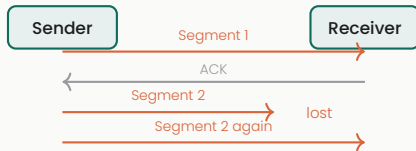


Make delivery reliable — when reliability is required

Packets can be lost, duplicated, corrupted, or arrive out of order.

A reliable transport protocol can detect and recover from these problems.

- Sequence numbers
- Acknowledgements
- Retransmission
- Ordering



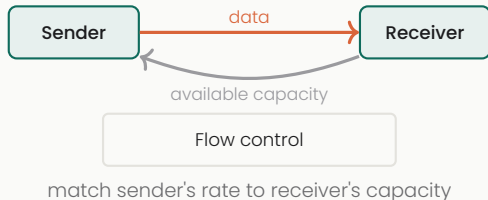
This is not a property of the network itself. It is a responsibility provided by a reliable transport protocol, such as TCP.

Do not overwhelm the receiver

Imagine the sender can transmit data much faster than the receiver can process it.

The receiver needs a way to say:

“Slow down — I can only take this much right now.”

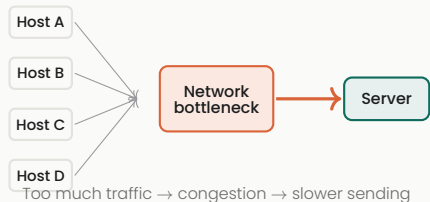


Do not overwhelm the network

Flow control protects the receiver.

But the network itself can become overloaded when too many senders transmit too much data.

Congestion control adapts the sending rate to network conditions.



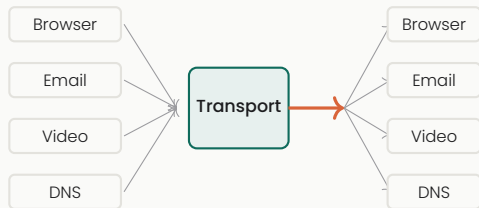
TCP uses congestion-control mechanisms to adjust its sending rate based on observed network conditions.

Combine many conversations — and separate them again

A computer may communicate with many applications and many remote systems simultaneously.

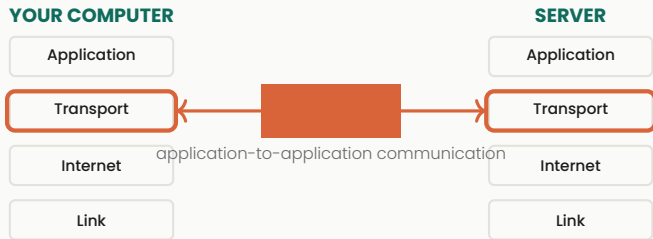
Transport multiplexes these conversations onto the network.

At the destination, it demultiplexes incoming segments and delivers each one to the correct application.



Multiplexing → Network → Demultiplexing

The server has a Transport layer too



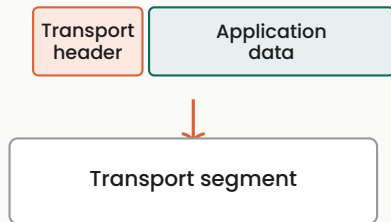
The Transport layer is not something that exists only on the sender's machine. The same layer exists at the server, where it receives transport segments and delivers their data to the appropriate server application.

Client Transport $\longleftrightarrow$ **Server Transport**

The application message gets a Transport header

The Application layer handed Transport a block of data. Transport now adds information needed to perform its job.

- Source port
- Destination port
- Sequence / control information
- Other protocol-specific fields



The resulting segment is passed down to the Internet layer, which will add its own information next.

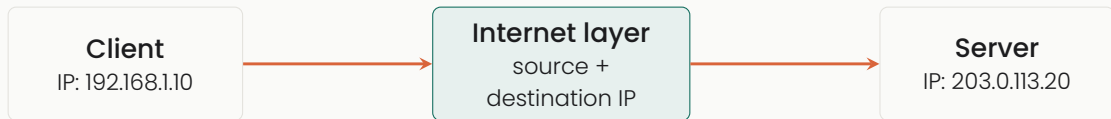
What Transport is responsible for

- **Application identification**
ports and application endpoints
- **Segmentation**
break and reassemble data
- **Reliability**
detect loss and recover when supported
- **Flow control**
protect the receiver
- **Congestion control**
adapt to network conditions
- **Multiplexing**
carry many conversations together
- **Demultiplexing**
deliver each conversation correctly

The big idea:

Transport is the layer that turns host-to-host communication

The Internet layer solves host-to-host delivery



Computer → Computer

The Internet layer uses IP addresses to identify hosts and move packets toward the destination host.

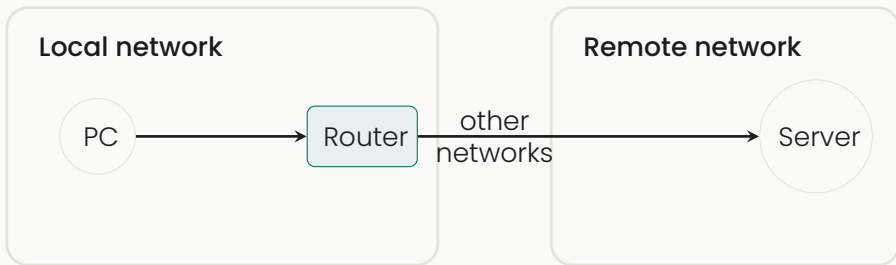
Identify the destination host

Every IP packet carries two important addresses.



IP address = host-level destination information

Is the destination on my network?

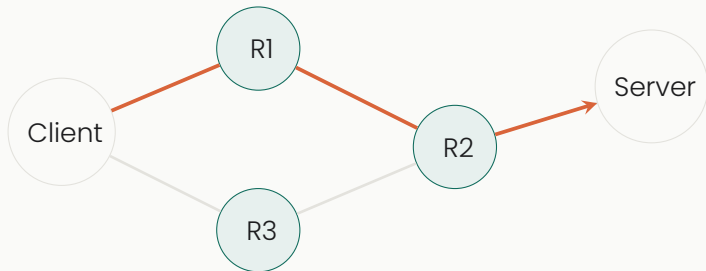


The destination IP address helps the host determine whether the destination is:

▶ **Local** — reachable directly on the local network.

▶ **Remote** — must be sent through a router.

Find a path through the network



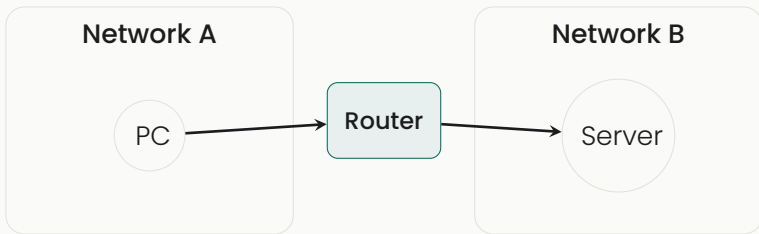
Routing = deciding where the packet should go next.

A packet may cross several different networks before reaching its destination.

NETWORKING MODELS

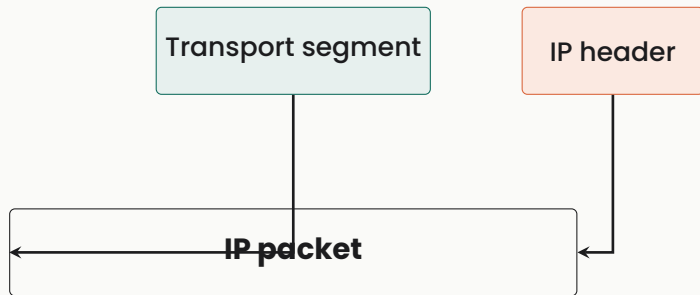
Routers examine the destination IP address and forward the packet toward the next hop.

Routers connect different networks



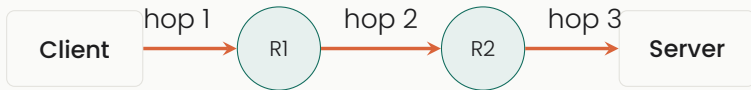
- ▶ A router has interfaces on different networks.
- ▶ It examines the destination IP address.
- ▶ It forwards the packet to the appropriate next hop.

Wrap the transport segment into an IP packet



NETWORKING MODELS The Internet layer adds information needed for host-to-host delivery, especially the source and destination IP addresses.

The packet does not jump directly to the server



Each router makes a local forwarding decision.

The packet keeps the same ultimate destination IP address while different routers forward it from one network to the next.

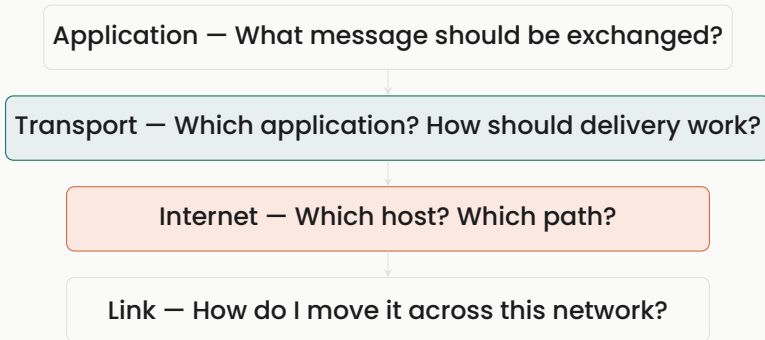
What has IP actually solved?

Host addressing

Routing across networks

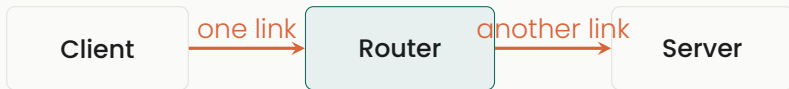
Packet forwarding

One message, different questions



Each layer answers a different question.

The packet has a destination. Now move it one hop.

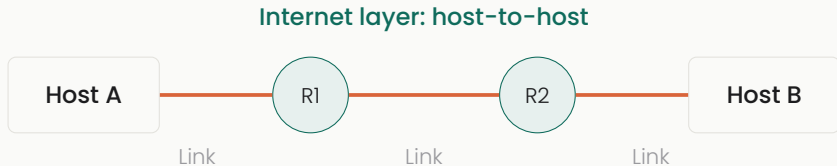


The Internet layer decides which host the packet should reach.
But the packet still needs to travel across the current network link.

The Link layer handles one-hop delivery.

THE BIG DISTINCTION

IP thinks end-to-end. Link thinks one hop.



Each individual link only needs to deliver the frame to the next device.

Identify the next device on the link

IP addresses identify hosts.
Link-layer addresses identify interfaces on the local link.



For a typical Ethernet/Wi-Fi network, this local addressing is represented by a MAC address.

Two layers, two kinds of addresses

IP address

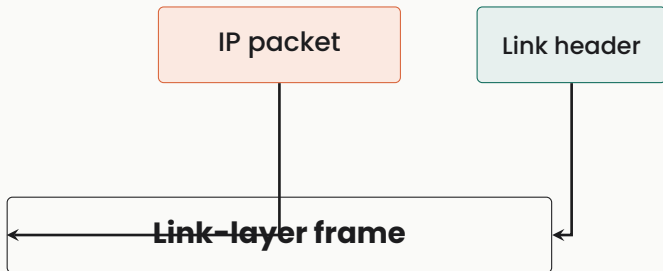
End-to-end
destination

MAC address

Local-link destination

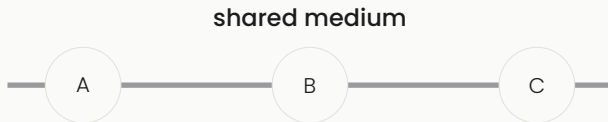
The IP destination can remain the same while the link-layer destination changes at every hop.

Turn the packet into a frame



The Link layer wraps the IP packet with information needed for delivery across the current link.

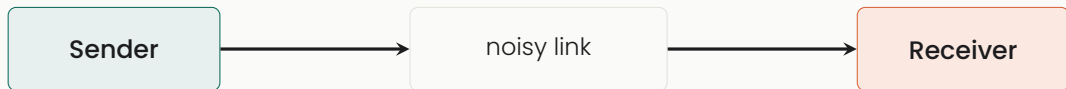
Who gets to use the shared link?



On some networks, multiple devices share the same communication medium. The Link layer defines how frames are placed onto that medium.

Who transmits? When? And how?

Did the frame arrive corrupted?

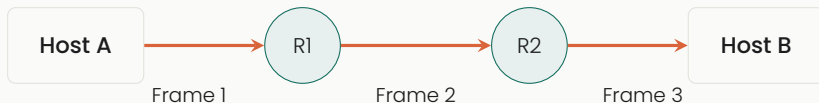


Frames can be affected by noise or interference while traveling over a physical link.

The Link layer can detect many transmission errors.

A common mechanism is a frame check sequence (FCS) or similar error-detection field.

The frame changes at every hop



At each router, the old link-layer frame is removed and the IP packet is placed into a new frame for the next link.

Same packet. New frame at each hop.

ENCAPSULATION

What changes as the packet travels?



IP information

Changes for each individual hop.

Link information

Mostly remains relevant to the end-to-end journey.

RECAP

What does the Link layer actually do?

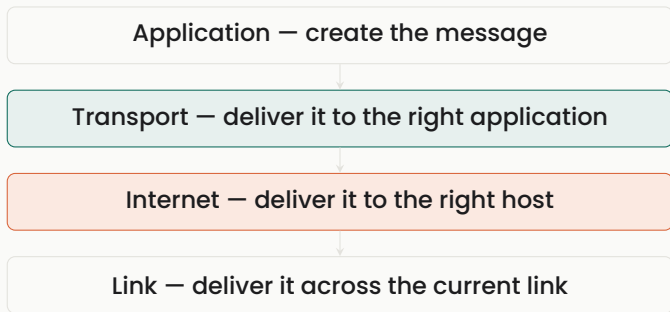
Local addressing

Framing

Medium access

Error detection

One message. Four different jobs.



Each layer solves a different part of the communication problem.

THE JOURNEY SO FAR

**A message becomes a packet,
then crosses the network one link
at a time.**

Application → Transport → Internet → Link

Now we can see why a networking model needs layers.

Application and Transport

Application

Provides network services to applications.

- ▶ Defines application-level communication
- ▶ Structures the data being exchanged
- ▶ Uses protocols such as HTTP, DNS, SMTP

Transport

Delivers data from one application to another.

- ▶ Identifies applications using ports
- ▶ Segments and reassembles data
- ▶ Provides reliability when required
- ▶ Controls flow and congestion

Internet and Link

Internet

Delivers packets from one host to another.

- ▶ Provides IP addressing
- ▶ Determines how packets are routed
- ▶ Forwards packets across networks

Link

Moves data across a single link.

- ▶ Uses local addressing such as MAC addresses
- ▶ Creates frames
- ▶ Controls access to the medium
- ▶ Detects transmission errors